

BUILD ON ENTIRE GRAPH

recipe-finder

Turn an end goal into a proven, *cited* integration recipe — grounded in how real repositories actually wire the code.

Say what you want to build. The code graph resolves which library real repos use, mines the real API calls, and returns a playbook where **every step has a real file:line receipt** — evidence, not a guess.

◆ Powered by the Entire Graph · deterministic · local · no-egress

Integration advice is *ungrounded*. Real code isn't.

THE QUESTION EVERY DEVELOPER ASKS

“I want to build X — which packages do I use, and how do I wire them together?”

Today the answer comes from blog posts, stale docs, and LLM guesses. None of it is grounded in what production code actually does — libraries drift, tutorials rot, and “plausible” APIs are often wrong.

THE IDEA

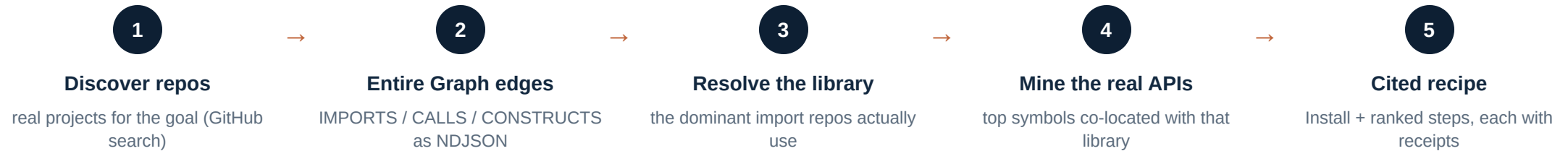
State the **end goal**, not a library. A precomputed **code graph** — functions, types, and the IMPORTS / CALLS / CONSTRUCTS relations between them — answers from reality.

The graph resolves **which library real repos truly use** and **how they wire it**, so the recipe is verifiable: every claim is an edge with a clickable receipt, not an opinion.

“8 of 10 collaborative-editor repos import `@tiptap/react`” is a fact you can click into — not a blog’s guess.

Goal in. *Cited playbook* out.

One engine drives the whole pipeline — discovery is grounded in reality, resolution and mining are grounded in the graph.



Verifiable by construction. The graph corrected our own design: for “kanban”, auto-resolution first surfaced an icon library — the IMPORTS edges showed real repos use `react-dnd`. Mining `@dnd-kit/sortable` then yielded `useSortable` / `useSensor` from real call sites, with receipts — the exact hooks a human would otherwise guess.

The graph is *evidence*, not an oracle.

Static analysis can't fully resolve dynamic dispatch, reflection, generated code, or inventory-only languages — and some files fail to parse. The product must never present incomplete evidence as certain. So every step is **graded**:

✓ Confirmed

The graph resolved this edge to a real definition / import. Safe default — still pin to your installed version.

≈ Heuristic · verify

Matched by name or inference — dynamic dispatch can fool it. Likely right; open a receipt and confirm against source.

? Unverified · check source

The graph couldn't resolve it — a blind spot, not a confirmed absence. Treat as a lead; confirm with source or a test.

Each recipe also carries an **analysis** block: it detects parse failures and blind repos, flags the result as **partial**, and prints a concrete **verification path**. A missing step is never treated as “not needed.”

One engine. *Three surfaces*. Scales out.

SURFACES

- › **Skill / CLI** — a coding agent pulls a recipe mid-task; the agent is the resolver.
- › **Web dashboard + API** — humans & programmatic callers; served from a mined corpus.
- › **Docs** — the verify-first angle, in the repo and on the site.

DEPTH

- › **Optional LLM resolver** — proposes candidate libraries; the graph still verifies.
- › **Databricks Spark path** — land raw edges, aggregate recipes in SQL across many repos.
- › **Evidence grading** + partial-analysis detection throughout.

Status. Live on Vercel (browse the mined corpus); the skill / local engine handles live mining. Language-agnostic engine; evidence-grading test suite green. Serverless can't run the parser + git — hosted deploy serves the corpus and returns an honest “live mining unavailable” for new goals.